

Полнотекстовый поиск SQLite в автономных приложениях

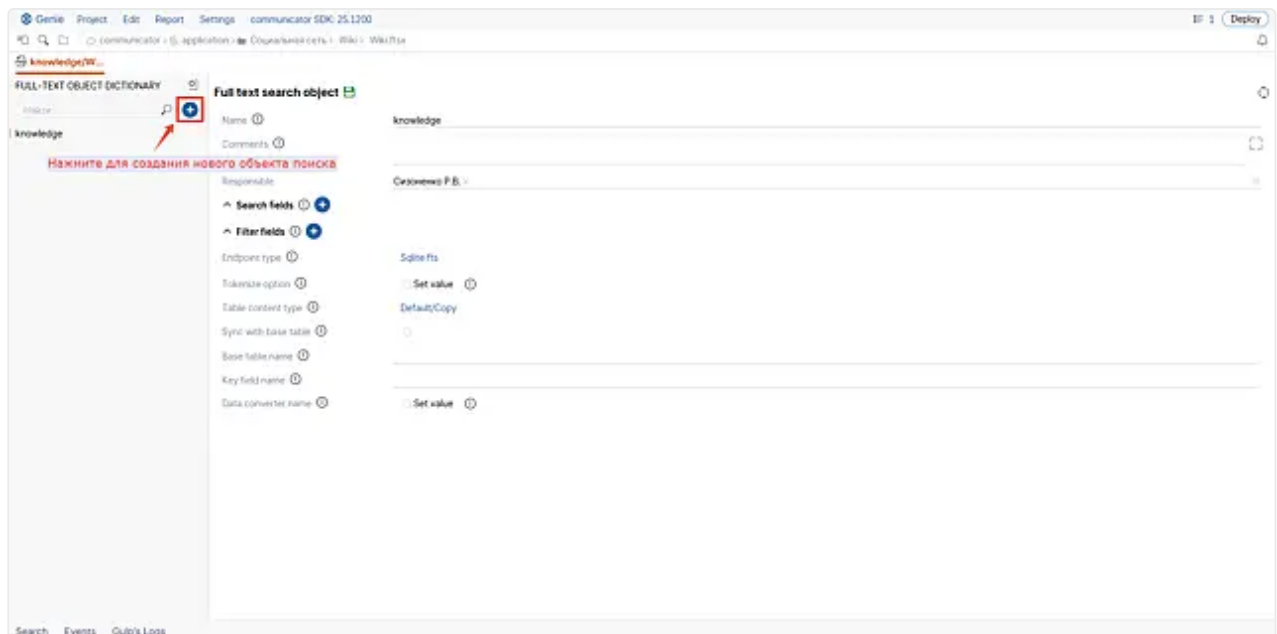
Для задач полнотекстового поиска (например, поиск в сообщениях или в базе знаний) в мобильных и оффлайн-приложениях используется решение на базе [SQLite FTS5](#).

В облаке аналогичный функционал реализован с помощью [Elasticsearch](#), однако он тяжелее и требует развертывание сервиса, что недопустимо или трудоемко для оффлайн-приложения.

Создание fts-индекса

Для создания fts-индекса SQLite выполните следующие шаги:

1. В [Genie](#) в модуле прикладного сервиса создайте файл для [описания поискового индекса \(ftsx\)](#). В атрибуте **Endpoint type** укажите значение "Sqlite fts".

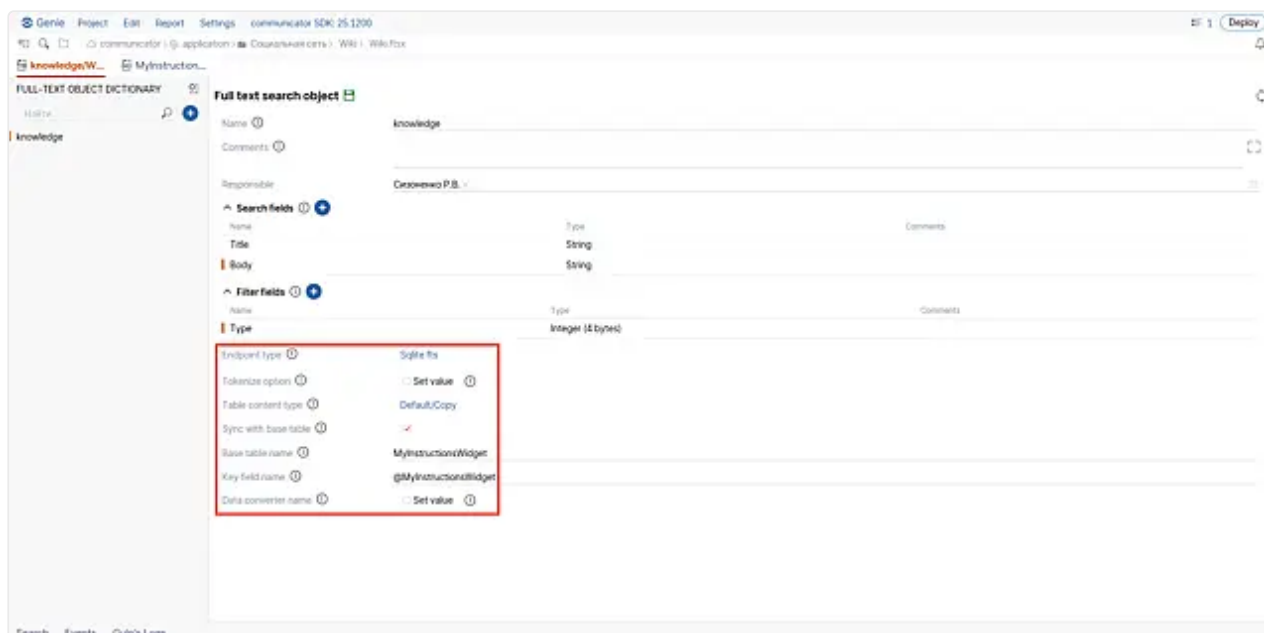


В настройке полей поиска для Sqlite fts параметры подсветки фрагмента текста не заполняются

- **Highlight fragments count**
- **Highlight fragment size**
- **Highlight no match size**

2. Укажите дополнительные параметры для вашего индекса (отображаются при выборе значения "Sqlite fts" в атрибуте **Endpoint type**).

- **Tokenize option** — значение параметра tokenize при создании fts5 индекса.
- **Table content type** — режим хранения данных
 - **Default/Copy** — по умолчанию. Представляет собой отдельную таблицу, которая хранит данные в себе.
 - **External content table** — через базовую таблицу, указанную в атрибуте **Base table name**. Применяется при большом объеме хранимых данных, которые нежелательно копировать.
- **Sync with base table** — нужно ли создавать триггеры синхронизации данных с базовой таблицей. Используется, если необходима автоматическая синхронизация данных.
- **Base table name** — имя [базовой таблицы](#).
- **Key field name** — имя поля из базовой таблицы, по которому выполняется синхронизация данных.
- **Data converter name** — имя конвертера, который будет выполнять обработку данных перед заполнением индекса. Конвертер должен быть объявлен с таким же именем и зарегистрирован в статическом инициализаторе своего модуля.



В результате при запуске приложения будет автоматически создан полнотекстовый индекс.

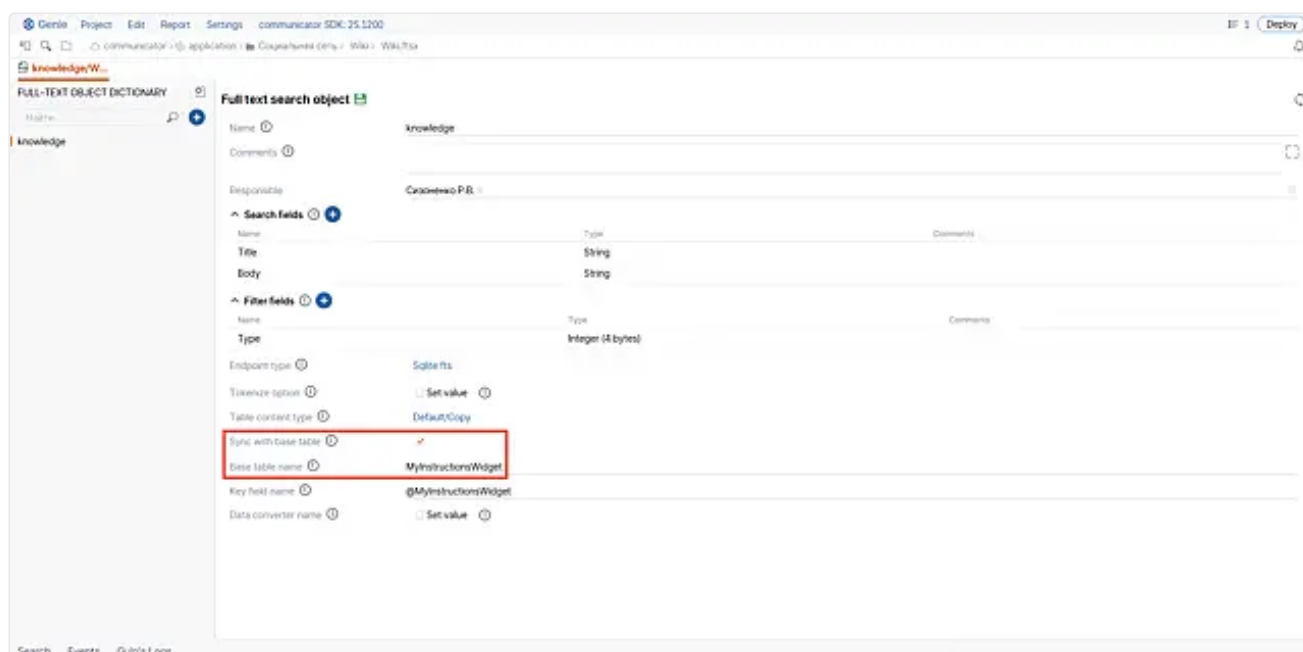
Поддержка целостности индекса с помощью триггеров

Для поддержки целостности данных можно использовать автогенерируемые триггерные функции.

У них существует два режима работы:

1. вставка данных выполняется "как есть" - при модификации базовой таблицы, модифицируются и данные поискового индекса;
2. перед вставкой в индекс вызывается пользовательский обработчик, написанный на C++, который может произвольно модифицировать данные для решения прикладных задач, например, привести все к нижнему регистру.

Для включения автоматической генерации триггеров поддержки целостности нужно отметить в Genie флаг **Sync with base table**, а также указать базовую таблицу (из disc-файла) и поле **rowid**. Затем конвертер БД сгенерирует все необходимые триггеры при запуске приложения.



Для использования пользовательского обработчика выполните следующие шаги:

1. Укажите имя обработчика в поле **Data converter name**.
2. В своем модуле в C++ коде создайте реализацию [ITriggerHandler](#):

```
1 class SBIS_FTS_SQLITE_PUBLIC ITriggerHandler
```

Имя хэндлера должно совпадать с данными, заполненными в поле **Data converter name**.

3. В [статической инициализации](#) модуля зарегистрируйте реализацию **ITriggerHandler** в методе **TriggerManager::Instance().AddHandler(...)**, который расположен в библиотеке sbis-fts-sqlite.

```
1 class FtsTriggerHandler : public fts_sqlite::ITriggerHandler
2 {
3 public:
4
5     StringView Name() const noexcept override
6     {
7         return L"my_handler_1"_sv;
8     }
9
10    void ProcessForInsert( const Record& /*row_new*/, Record&
index_row_new ) const override
11    {
12        String& body = index_row_new.Ref< String >( L"Body" );
13        to_lower( &body[ 0 ], body.size() );
14    }
15
16    void ProcessForUpdate( const Record& /*row_old*/, Record&
index_row_old,
17                          const Record& /*row_new*/, Record&
index_row_new ) const override
18    {
19        {
20            String& body = index_row_new.Ref< String >( L"Body" );
21            to_lower( &body[ 0 ], body.size() );
22        }
23        {
24            String& body = index_row_old.Ref< String >( L"Body" );
25            to_lower( &body[ 0 ], body.size() );
26        }
27    }
28 };
```



Внутри функции обработки вызовы в облако и в БД делать нельзя!

Обработчик должен быть максимально легковесным и выполнять работу быстро.